Correzione della Verifica 1: if, if-else e condizioni

Lezione 11 — 1IOT, SST-ICT-01 — Ripasso del costrutto if, la condizione, l'esercizio dello sconto, parentesi, scrittura su più righe, confronto con Python

13/10/2026

1 Da dove ripartiamo

Nota

Questa lezione nasce dalla **correzione della Verifica 1** del 12/10. Due punti hanno creato più difficoltà e vanno ripresi:

- la formula del **numero di combinazioni** in binario;
- il **costrutto `if`**, e in particolare che cosa si scrive nella **condizione**.

Il docente lo dice chiaramente: l'`if` si usa "**dalla mattina alla sera**, tutti i giorni". Non averlo capito blocca **qualunque altro argomento** che faremo dopo. Per questo, prima di andare avanti, si fa un passo indietro e si rimette ordine. Tutto questo è già nelle dispense precedenti (A08 e A09), ma va **rivisto** perché in verifica molti l'hanno sbagliato.

2 Il numero di combinazioni: 2^n

Definizione

Con n **lampadine** (cioè n bit), ognuna accesa o spenta, il **numero di combinazioni** possibili è

$$\text{combinazioni} = 2^n$$

Per esempio: 1 lampadina dà $2^1 = 2$ combinazioni, 3 lampadine danno $2^3 = 8$, 8 lampadine (un byte) danno $2^8 = 256$. Questa formula va **sottolineata** sul quaderno: è quella su cui si è sbagliato di più in verifica.

3 Il costrutto if: fare o non fare

Il costrutto **`if`** è un **costrutto decisionale**: permette alla macchina di **scegliere** se eseguire un pezzo di codice oppure no. Ci sono due strutture, **sorelle** tra loro.

Definizione

La struttura `if`. Tra le **parentesi tonde** si scrive una **condizione**; tra le **graffe** il codice da eseguire. Alla lavagna il docente usa tre colori: la condizione in **rosso**, il codice dell'`if` in **verde**.

```
if (condizione)
{
    printf("...");     // dentro ci puo' essere qualunque codice:
    z = 3 + 5;         // una stampa, un calcolo, altro ancora
}
```

La parte in verde viene eseguita **solamente se** la condizione in rosso viene valutata come **vera**. Se è falsa, il programma la salta e prosegue.

Definizione

La struttura `if-else`. È la stessa cosa, con in più un secondo blocco:

```
if (condizione)
{
    // codice "verde": eseguito se la condizione e' VERA
}
else
{
    // codice "blu": eseguito se la condizione e' FALSA
}
```

Esempio

Quale scegliere? Le due strutture sono molto simili e non vanno imparate come due cose separate: si usa l'una o l'altra in base a cosa si deve fare.

- Devo **fare o non fare** qualcosa? Basta l'**`if`**.
- Devo **scegliere tra due** operazioni diverse? Serve l'**`if-else`**.

Attenzione

Vero e falso, non positivo e negativo. Un dubbio emerso in classe: "in verifica ho ragionato in *positivo* e *negativo* e mi sono confuso". Le parole positivo e negativo fanno pensare a un **verso di percorrenza** del diagramma di flusso. Nella condizione, invece, si parla solo di **vero** o **falso**: è questo il **passaggio concettuale** da sistemare.

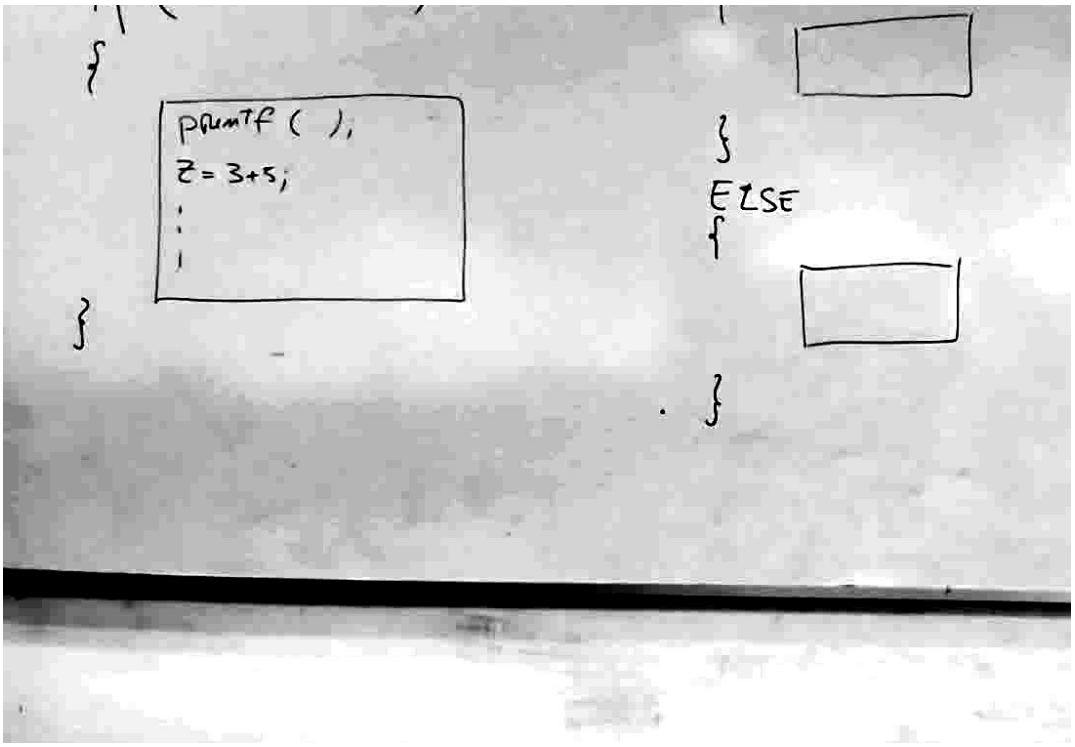


Figura 1: La lavagna all'inizio della lezione: a sinistra la struttura `if` con un esempio di codice nel blocco (una `printf` e un calcolo), a destra la struttura `if-else` con i due blocchi.

4 Che cosa si scrive nella condizione

Definizione

La **condizione** è una qualunque **espressione** che dia come risultato **vero o falso**. Il caso più comune è un **confronto** tra due valori, con gli operatori di confronto già visti (==, !=, >, >=, <, <=).

Esempio

Gli esempi alla lavagna.

- `a == b`: **uguaglianza** tra due variabili. Se `a` contiene 5 e `b` contiene 5, il risultato è **vero**; se `a` vale 5 e `b` vale 7, è **falso**.
- `a > b`: se `a` vale 6 e `b` vale 5 è evidentemente vero, perché 6 è più grande di 5. Ma se `a` vale 5 e anche `b` vale 5? È **falso**: 5 non è maggiore di 5.
- `a >= b`: per avere **vero** anche quando i due valori sono uguali si usa il **maggiore o uguale**, che "mette nel paniere" anche la possibilità che siano uguali. Con `a` = 5 e `b` = 5 è vero.

Un caso al limite: con `b` = 5 e `a` = 5,0000001 i due numeri sono **diversi**, quindi `a > b` è vero (e naturalmente anche `a >= b`).

Nota

Accanto a maggiore e maggiore o uguale ci sono, allo stesso modo, **minore** (<) e **minore o uguale** (<=). Non serve matematica di alto livello, ma **maggiore e minore** vanno padroneggiati: è un problema, ma **risolvibile**.

5 L'esercizio della verifica: lo sconto

Esempio

Testo. Un utente **avrà lo sconto** se ha **meno di 12 anni** **oppure** se ha **65 anni o più**.

Passo 1: scomporre la frase

Si scompone la frase nei suoi simboli. Ci sono **due condizioni**:

- età **minore di 12**: `eta < 12`;
- età **maggiore o uguale a 65**: `eta >= 65` (alla lavagna, sotto "maggiore", il docente aggiunge "uguale").

Nel codice la variabile si chiama `eta`, **senza accento**: alla lavagna è scritta "età", ma nei nomi delle variabili gli accenti non sono ammessi (A09). Le due condizioni sono legate dalla parola **"oppure"**, cioè da una logica di tipo **OR**.

Attenzione

Come si scrive l'OR. L'OR si scrive con **due barre verticali**: `||`. **Non** è lo slash (`/`) e non è una barra sola. La barra verticale in inglese si chiama ***pipe***: è il nome che si usa anche nei sistemi Unix, dove fa da separatore tra comandi. In classe il docente si è detto non del tutto sicuro del nome: è corretto.

Passo 2: la condizione completa

```c
if ((eta < 12) || (eta >= 65))
{
    printf("Sconto");
}
else
{
    printf("No sconto");
}
```

Definizione

Perché le parentesi tonde intorno a ogni condizione. Al docente piace mettere ogni condizione tra **tonde**: così l'espressione è **"blindata"** e l'ordine in cui vengono valutati gli operatori non può essere frainteso.

In questo esercizio, a dire il vero, si potrebbe anche scrivere `if (eta < 12 || eta >= 65)` e funzionerebbe lo stesso: non ci sono altri modi sensati di leggere l'espressione. Le parentesi si mettono comunque, perché chi legge capisce subito e non si rischia di sbagliare.

Attenzione

Parentesi e portabilità. In classe il docente ha motivato le parentesi anche dicendo che "ci sono compilatori che cambiano le priorità". Precisazione: la **priorità degli operatori** in C è **fissata dallo standard** del linguaggio (lo si è detto anche in A10), quindi un compilatore corretto non la cambia. Resta vero il discorso generale sulla **portabilità**: lo stesso codice compilato su un'altra macchina (PC, Mac, Linux, Arduino) usa un altro compilatore e un altro processore, e alcune cose possono cambiare (per esempio la dimensione di un `int`). La regola pratica non cambia: **parentesi sempre** nelle condizioni composte.

Un'altra soluzione: due if

In verifica qualcuno ha risolto con **due `if`**: prima controlla se l'età è minore di 12; se non lo è, nell'`else`, controlla se è almeno 65.

```
if (eta < 12)
{
    printf("Sconto");
}
else
{
    if (eta >= 65)
    {
        printf("Sconto");
    }
    else
    {
        printf("No sconto");
    }
}
```

Funziona: l'età o è minore di 12, o è almeno 65, e non può mai essere tutte e due le cose insieme. Il docente la considera **risolta bene**. La soluzione con l'OR è però più **compatta**: un solo `if`, una sola condizione. (Il blocco `else` interno, con "No sconto", è completato qui per mostrare il programma intero; alla lavagna era abbozzato.)

6 Scrivere la condizione su più righe

Domanda dalla classe: è sempre necessario scomporre tutto e mettere le parentesi? E si può andare a capo dentro una condizione lunga?

- Al C **non interessano gli spazi** e gli a capo: la condizione si può anche spezzare su più righe, e l'espressione resta la stessa.
- **Non** si fa invece a colpi di tabulazione a caso: si rovina l'**indentazione** del resto del codice.

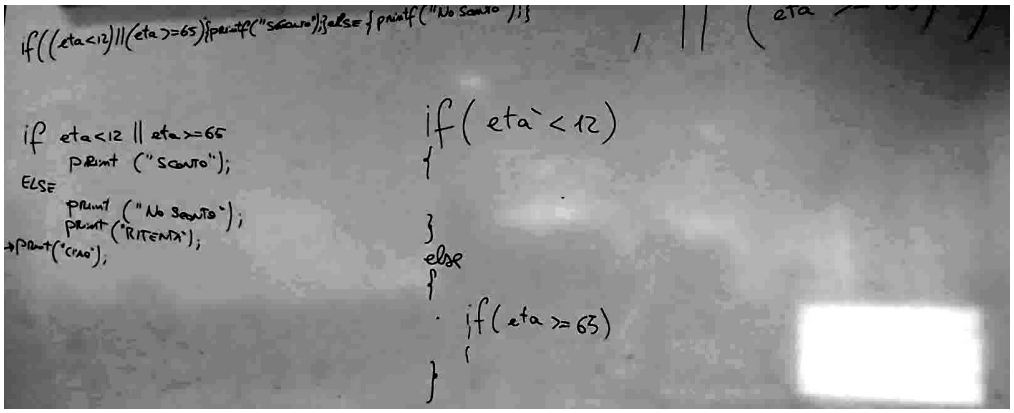


Figura 2: La lavagna a fine lezione. In alto a sinistra: tutto l'`if-else` dello sconto scritto su una sola riga. Sotto: la stessa logica "alla Python", senza graffe, con il rientro. A destra: la soluzione con due `if`, il secondo dentro l'`else` (il valore che sembra "63" va letto 65).

```
if (
    (eta < 12)
    ||
    (eta >= 65)
   )
```

Nota

Con due o tre condizioni scriverle così è **esagerato**: una riga sola va benissimo. Ha senso pensarci quando le condizioni diventano **tante**. Per esempio: sconto se ha meno di 12 anni, oppure più di 65, oppure 42, oppure 47, oppure 32, oppure ha i capelli biondi... In quei casi si tende a **raccogliere le condizioni** da un'altra parte e nel codice usare solo un **nome** (un'etichetta) che le rappresenta: è un argomento che vedremo più avanti.

Attenzione

Tutto su una riga si può, ma non si fa. Siccome al C non interessano gli a capo, si potrebbe scrivere l'intero `if-else` su **una sola riga**:

```
if((eta<12)||(eta>=65)){printf("Sconto");}else{printf("No sconto");}
```

Il compilatore lo accetta, ma nessuno lo fa: **è illeggibile**, e quando c'è un errore il compilatore segnala la riga, che qui è una sola, e trovarlo diventa difficile. Si scrive **indentato**, come nei listati sopra.

7 Confronto con Python: graffe o rientro

Una domanda sul **Python**. Il docente premette con onestà che il Python non è un linguaggio che usa abitualmente; la differenza che interessa qui è una sola.

- In **C** i limiti del blocco dell'`if` li segnano le **graffe**.
- In **Python** non ci sono graffe: è il **rientro** (l'indentazione) a dire quale codice fa parte dell'`if` o dell'`else`.

```
if eta < 12 or eta >= 65:
    print("Sconto")
else:
    print("No sconto")
    print("Ritenta")
print("Ciao")
```

Le due istruzioni rientrate sotto l'`else` ("No sconto" e "Ritenta") vengono eseguite solo se la condizione è falsa, come se fossero tra graffe. La `print("Ciao")`, allineata all'`if`, **non c'entra niente** con l'`if`: viene eseguita **in qualunque caso**. Secondo il docente questo è un po' più confuso: la **graffa segna i limiti** entro cui agisce il blocco, in Python lo fa solo il rientro.

Nota

Nota di correzione. Alla lavagna l'esempio Python era scritto in modo schematico, con `||` e il punto e virgola. In Python vero l'OR si scrive con la parola `or`, dopo la condizione e dopo `else` ci vanno i **due punti**, e il punto e virgola finale non serve. Il listato qui sopra è scritto nella forma corretta.

8 Come proseguiremo

Nota

- Si continua con il **C**: i costrutti fondamentali e gli esempi si fanno in C.
- Più avanti si farà un po' di **C++**, trattato praticamente come se fosse C.
- L'obiettivo è che a **fine anno** sappiate scrivere da soli un codice, non particolarmente complesso, ma **scritto da voi**. Tutto quello che vediamo adesso ce lo portiamo dietro: aumenteranno solo le cose che si possono fare.

9 La consegna delle verifiche corrette

Le verifiche corrette verranno consegnate **in modo individuale**: ciascuno riceverà solo la propria. Il docente sta valutando come farlo (per posta elettronica o nei commenti privati di Classroom) e se automatizzare l'invio, per non rischiare di scambiare le verifiche tra due persone. Alla verifica potrebbe essere allegato anche un **commento** con le cose da rivedere, così quello che è stato detto a voce resta anche per iscritto. L'idea è che il rapporto tra docente e classe sia il più **trasparente** e completo possibile.

10 Cosa ricordare di questa lezione

- Con n lampadine (bit) le **combinazioni** sono 2^n.
- **`if`**: fare o non fare un blocco. **`if-else`**: scegliere tra due blocchi.
- La **condizione** è un'espressione che vale **vero o falso**, non "positivo o negativo".
- `>` esclude l'uguaglianza, `>=` la include: con 5 e 5, `a > b` è **falso**, `a >= b` è **vero**.

- “Oppure” si scrive `||`, due barre verticali (*pipe*), non lo slash.
- Sconto: `if ((eta < 12) || (eta >= 65))`; **parentesi sempre** intorno a ogni condizione.
- Due `if` annidati **funzionano**, l’OR è più **compatto**.
- In C le **graffe** delimitano i blocchi; in **Python** lo fa il rientro.